

Università degli Studi di Verona

Corso di Laurea in Informatica

MusicManager

Documentazione Tecnica del Progetto

*Analisi, Progettazione e Implementazione
di un Sistema Software Orientato agli Oggetti*

Autori:

Angelo Vaccaro

Alex Zanoni

Omar Fedior

Matricola: VR480616

Matricola: VR523734

Matricola: VR520777

Anno Accademico: 2025/2026

Indice

1	Introduzione	3
1.1	Come è organizzato il documento	3
2	Scelte Architettureali di Base	4
2.1	Interfaccia grafica con JavaFX e separazione MVC	4
2.2	Riproduzione dei file: system call invece di librerie interne	4
2.2.1	Segmentazione temporale per i concerti	4
2.3	Gestione della sessione con il pattern Singleton	5
2.4	Persistenza con JSON e Google Gson	5
2.5	Controllo degli accessi (RBAC) e dizionari tassonomici	5
2.6	Registrazione degli utenti e pipeline di validazione	6
3	Analisi dei Requisiti e Modellazione dei Casi d'Uso	7
3.1	Gli attori del sistema	7
3.2	Descrizione funzionale dei casi d'uso	7
3.2.1	Area Accessi e Sicurezza	7
3.2.2	Area Gestione Catalogo Tassonomico	7
3.2.3	Area Interazione Social e Multimedialità	8
3.3	Diagramma dei Casi d'Uso	9
3.4	Giustificazione delle relazioni UML	9
3.4.1	Relazioni di Inclusione («include»)	9
3.4.2	Relazioni di Estensione («extend»)	10
3.5	Schede di Specifica dei Casi d'Uso	10
3.5.1	UC-01: Inserire Nuovo Brano	10
3.5.2	UC-02: Ricercare Contenuti	10
3.5.3	UC-03: Aggiungi Commento/Risposta	11
3.5.4	UC-04: Gestione e Aggiornamento Dizionari	11
3.5.5	UC-05: Richiedere Registrazione	12
3.5.6	UC-06: Valutare Richieste	12
4	Modellazione Concettuale e Progettazione Statica	14
4.0.1	Come siamo arrivati al diagramma definitivo	14
4.1	Il Class Diagram	15
4.2	Analisi delle classi per package	15
4.2.1	Package utenti (<code>com.musicmanager.model.users</code>)	15
4.2.2	Package catalogo (<code>com.musicmanager.model.catalog</code>)	16
4.2.3	Package feedback (<code>com.musicmanager.model.feedback</code>)	16
4.3	Design Pattern strutturali	17
4.3.1	Pattern Composite — commenti annidati	17

4.3.2	Pattern Iterator — scorrere la Playlist	17
4.3.3	Pattern Singleton — gestione della sessione	18
4.4	Riferimenti circolari e chiavi logiche	18
4.4.1	Il problema	18
4.4.2	La soluzione: chiavi logiche	18
5	Modellazione Dinamica e Comportamentale	19
5.1	Diagrammi di Sequenza	19
5.1.1	SD-01: Inserimento di un nuovo brano	19
5.1.2	SD-02: Inserimento di un commento o risposta	20
5.1.3	SD-03: Ricerca avanzata con eccezione custom	21
5.1.4	SD-04: Approvazione di una richiesta di registrazione	22
5.2	Diagrammi delle Attività	23
5.2.1	AD-00: Ciclo di vita globale del sistema	23
5.2.2	AD-01: Flusso di login	24
5.2.3	AD-02: Pipeline di registrazione	25
5.2.4	AD-03: Salvataggio e modifica di un brano	27
5.2.5	AD-04: Ricerca con deep scanning ed eccezione custom	28
6	Implementazione, Collaudo e Note Tecniche	30
6.1	Uso delle Java Stream API	30
6.2	Test unitari con JUnit 5	30
6.3	System Test funzionali	32
6.4	Documentazione Javadoc	33
A	Manuale d'Installazione e Note di Esecuzione	34
A.1	Struttura dell'archivio	34
A.2	Prerequisiti	35
A.3	Come avviare il progetto	35
A.3.1	Avvio da riga di comando (CLI)	35
A.3.2	Avvio da IDE	35
A.4	Credenziali per il collaudo	36

Capitolo 1

Introduzione

Questo documento descrive come abbiamo analizzato, progettato e implementato **MusicManager**, un'applicazione Java per gestire, catalogare e condividere contenuti musicali di vario tipo. Tutto il lavoro è stato organizzato seguendo un approccio *Top-Down*: siamo partiti dai requisiti ad alto livello e siamo scesi fino al codice Java vero e proprio, cercando di mantenere una traccia chiara di ogni scelta lungo il percorso.

L'idea di base è che MusicManager non sia un semplice archivio di file sul disco, ma qualcosa di più strutturato: un sistema che modella i *Brani* come entità logiche, associandoli a file multimediali di diversi formati (PDF per gli spartiti, MP3, MP4, link YouTube) e permettendo agli utenti di interagirci in modo collaborativo.

Le due sfide principali che abbiamo dovuto affrontare sono state la gestione temporale dei media e il sistema di commenti annidati. Per la prima avevamo bisogno di poter “spezzare” un file multimediale (per esempio il video di un concerto) in segmenti, ognuno corrispondente a un brano specifico, con tanto di timestamp di inizio e fine e note sulla performance. Per la seconda, i commenti devono poter contenere risposte, le risposte possono avere altre risposte, e così via senza un limite fisso di profondità: per questo abbiamo usato il pattern Composite, che si prestava bene al problema.

Per tenere tutto ordinato, abbiamo seguito l'architettura *Model-View-Controller* (MVC) con JavaFX e applicato alcuni design pattern classici dove aveva senso farlo, cercando di rispettare i principi SOLID senza esagerare con l'astrazione.

1.1 Come è organizzato il documento

Il resto della relazione segue più o meno l'ordine in cui abbiamo lavorato al progetto:

2. **Scelte Architettureali di Base:** le decisioni tecnologiche principali, come funziona la persistenza e come gestiamo la sicurezza.
3. **Analisi dei Requisiti e Casi d'Uso:** cosa deve fare il sistema, chi lo usa e come, con le schede UML.
4. **Modellazione Concettuale e Progettazione Statica:** il Class Diagram con tutte le classi, le relazioni e i pattern.
5. **Modellazione Dinamica e Comportamentale:** come si comporta il sistema a runtime, attraverso Sequence Diagram e Activity Diagram.
6. **Implementazione, Collaudo e Note Tecniche:** le scelte implementative più interessanti, i test JUnit e il Javadoc.

Capitolo 2

Scelte Architettureali di Base

Prima di scrivere una riga di codice abbiamo preso alcune decisioni strutturali che poi hanno condizionato tutto il resto. Le descriviamo qui, cercando di spiegare anche il perché di ogni scelta.

2.1 Interfaccia grafica con JavaFX e separazione MVC

Per la UI abbiamo scelto **JavaFX**. Rispetto a Swing, ci sembrava più moderno e soprattutto spinge naturalmente verso una separazione netta tra logica e presentazione grazie ai file **.fxml**, che descrivono il layout in XML senza mischiarlo con il codice Java. Questo si sposa bene con il pattern MVC: il *Model* contiene le classi del dominio (Brano, Utente, Commento, ecc.), la *View* è dichiarata nei file FXML e lo Scene Graph se ne occupa a runtime, i *Controller* intercettano gli eventi dall'interfaccia e coordinano le modifiche al modello e alla persistenza. Questa separazione ci ha anche aiutato nei test, perché la logica di business è indipendente dalla UI.

2.2 Riproduzione dei file: system call invece di librerie interne

Quando l'utente vuole aprire uno spartito PDF o riprodurre un MP3, non abbiamo voluto caricare librerie pesanti di decodifica dentro la JVM. Abbiamo preferito delegare tutto al sistema operativo, sia per mantenere l'applicazione leggera sia per evitare di gestire buffer multimediali o parser potenzialmente vulnerabili.

La classe astratta `DocumentoDigitale` dichiara il metodo astratto `apriConPlayerEsterno()`, e ogni sottoclasse concreta (`TracciaAudio`, `TracciaVideo`, `Spartito`, `CollegamentoWeb`) lo implementa usando `java.awt.Desktop` per aprire il file con il programma predefinito dell'utente (VLC, il browser, Adobe Reader, ecc.). L'applicazione risulta così cross-platform senza dover gestire internamente alcun formato multimediale.

2.2.1 Segmentazione temporale per i concerti

Per supportare file di lunga durata come la registrazione di un concerto, abbiamo aggiunto la classe `SegmentoEsecuzione`. L'idea è che un singolo `DocumentoDigitale` possa essere “diviso” logicamente in più segmenti, ognuno con timestamp di inizio e fine in `float` (così si possono gestire anche frazioni di secondo), metadati del segmento (titolo del brano eseguito, nome dell'esecutore, strumenti usati) e un campo testo libero per commenti qualitativi sull'esecuzione. La relazione tra `DocumentoDigitale` e `SegmentoEsecuzione` è di tipo 0..*: per i file normali non ci sono segmenti, ma per un

concerto se ne possono aggiungere quanti ne servono.

2.3 Gestione della sessione con il pattern Singleton

Per tenere traccia dell'utente attualmente loggato e rendere questa informazione accessibile da qualsiasi parte dell'applicazione, abbiamo usato il design pattern Singleton nella classe `SessionManager`. Il costruttore è privato, l'unico punto di accesso è il metodo statico `getInstance()`, e l'oggetto viene creato solo alla prima chiamata (lazy initialization). Così siamo sicuri che in tutta l'applicazione esista un unico `SessionManager`, che mantiene il riferimento all'utente loggato e permette controlli di sicurezza rapidi prima di operazioni sensibili.

2.4 Persistenza con JSON e Google Gson

Abbiamo scelto di persistere i dati su file JSON usando la libreria **Google Gson** per la serializzazione/deserializzazione. La classe `GestoreJSON` gestisce tutto il ciclo di lettura e scrittura. Durante lo sviluppo abbiamo incontrato alcune insidie che vale la pena documentare.

La prima riguarda la posizione dei file JSON: `catalogo.json`, `utenti.json` e `dizionari.json` stanno nella root del progetto, *non* dentro la cartella `resources`. Se finissero dentro il JAR compilato da Maven diventerebbero read-only, e l'applicazione crasherebbe al primo tentativo di scrittura. Lo abbiamo scoperto a nostre spese durante i test iniziali, quando la prima esecuzione del JAR produceva un'eccezione non gestita alla prima operazione di salvataggio. La seconda riguarda il polimorfismo: poiché il formato JSON non rappresenta nativamente l'ereditarietà, il costruttore di `GestoreJSON` registra degli adattatori gerarchici personalizzati (`registerTypeHierarchyAdapter`) per le gerarchie di `DocumentoDigitale`, `UtenteAutorizzato` e `Commento`. In fase di scrittura ogni oggetto riceve un campo discriminante (`typeClass` per i documenti, `type` per utenti e commenti) che ne traccia il tipo concreto; in fase di lettura quello stesso campo permette di ricostruire la sottoclasse esatta (ad esempio `TracciaAudio` invece del generico `DocumentoDigitale`, oppure `NotaTemporale` invece di `Commento`). Le routine di caricamento usano infine *try-with-resources* per garantire la chiusura dei file anche in caso di eccezione, intercettando in modo difensivo sia gli errori di I/O sia i casi di JSON mancante o malformato.

2.5 Controllo degli accessi (RBAC) e dizionari tassonomici

Il sistema implementa una politica di controllo degli accessi basata sui ruoli (RBAC). Al momento del login, l'istanza dell'utente viene salvata nel `SessionManager`. Quando si carica la Dashboard, il sistema controlla con `instanceof` se l'utente è un `Amministratore`: in caso affermativo, vengono aggiunti alla UI i pulsanti per la gestione utenti e i dizionari (`btnGestioneUtenti` e `btnGestioneDizionari`), che altrimenti non vengono nemmeno allocati.

Per mantenere la coerenza del database, il sistema gestisce quattro dizionari controllati che solo l'amministratore può modificare: generi musicali, strumenti, titoli delle opere e nomi degli autori. In fase di inserimento di un brano, il genere e l'autore principale devono obbligatoriamente appartenere ai rispettivi dizionari (`Generi` e `Autori`): per

questi due campi non è ammesso testo libero. La ricerca, invece, è testuale e multi-criterio, ma trae vantaggio dalla terminologia uniforme garantita dai dizionari, evitando il problema delle varianti di battitura (“jazz”, “Jazz”, “JAZZ”) che frammenterebbero il catalogo rendendolo difficile da interrogare.

2.6 Registrazione degli utenti e pipeline di validazione

Il controller di registrazione (`RegistrazioneController`) controlla i dati inseriti su tre livelli prima di scrivere qualcosa su disco. Prima verifica che nessun campo obbligatorio sia vuoto o nullo; poi, usando le Stream API, scansiona la lista degli utenti esistenti per garantire l’unicità dell’email (analogo al vincolo `UNIQUE` di un database relazionale); infine assegna a ogni nuovo utente un ID generato con `java.util.UUID`, così non ci sono mai collisioni di identificativi.

Una volta superati questi controlli, l’istanza viene creata polimorficamente — si distingue tra `UtenteAutorizzato` standard e `AutoreInterprete` in base al ruolo scelto — e salvata con il flag `attivo = false`: non potrà fare login finché l’amministratore non la approva esplicitamente.

Capitolo 3

Analisi dei Requisiti e Modellazione dei Casi d'Uso

In questo capitolo definiamo cosa deve fare MusicManager, chi lo usa e come. Abbiamo usato il formalismo dei Casi d'Uso UML 2.0 per mappare i requisiti in modo sistematico.

3.1 Gli attori del sistema

Abbiamo identificato quattro attori, organizzati in una gerarchia di permessi crescente. Il **Visitatore** (Guest) è l'utente non ancora autenticato: può solo chiedere di registrarsi o tentare il login, senza poter fare altro finché non viene approvato. L'**Utente Autorizzato** è l'utente base approvato dall'admin: può inserire brani, associare file multimediali, aprire i player esterni e interagire con commenti e risposte. L'**Autore/Interprete** è una specializzazione dell'Utente Autorizzato (relazione di generalizzazione UML): eredita tutto ciò che può fare l'utente base, ma in più può aggiungere informazioni sulla propria attività artistica (strumenti, biografia) e inserire commenti "ufficiali" sui brani che lo riguardano, evidenziati graficamente in modo diverso. L'**Amministratore di Sistema**, infine, gestisce gli account (approva, rifiuta, elimina), mantiene i dizionari tassonomici e può moderare qualsiasi commento nel sistema.

3.2 Descrizione funzionale dei casi d'uso

Abbiamo raggruppato le funzionalità in tre aree principali.

3.2.1 Area Accessi e Sicurezza

Quest'area comprende le operazioni di autenticazione e gestione degli account. Il Visitatore può compilare il form di registrazione con nome, cognome, email, password ed eventuale biografia artistica. Al login il sistema verifica le credenziali e controlla che l'account sia attivo prima di aprire la Dashboard; il logout invalida la sessione in RAM e riporta l'interfaccia alla schermata iniziale. Solo l'Admin può esaminare le iscrizioni in sospenso (approvandole o rifiutandole) e rimuovere definitivamente un account dal database.

3.2.2 Area Gestione Catalogo Tassonomico

- **Inserire Nuovo Brano:** aggiunge un brano all'archivio con titolo, autore e anno. Il genere musicale deve essere scelto tra quelli nei dizionari di sistema, non si può scrivere testo libero.

- **Ricerca Contenuti:** filtra i brani in base a genere, titolo, autore o esecutore. La ricerca è multi-criterio e include anche la scansione dei segmenti di esecuzione nei file associati.
- **Filtrare I Miei Contributi:** mostra solo i brani su cui l'utente ha commentato o aggiunto note temporali.
- **Gestione e Aggiornamento Dizionari:** funzionalità esclusiva dell'Admin per aggiungere nuove voci ai quattro dizionari.

3.2.3 Area Interazione Social e Multimedialità

- **Visualizzare Dettaglio Brano:** apre la scheda di dettaglio di un brano, da cui si accede ai commenti, ai file associati e alle funzionalità multimediali.
- **Associa File Multimediale:** collega un file fisico (MP3, MP4, PDF) o un URL al brano selezionato.
- **Riproduci Media:** apre il file con il player predefinito del sistema operativo.
- **Aggiungi Commento/Risposta:** permette di scrivere feedback testuali e di rispondere ad altri commenti, senza limiti di profondità di annidamento (pattern Composite).
- **Aggiungi Nota Temporale:** aggiunge un'annotazione testuale collegata a un intervallo di tempo preciso (in secondi) di un file audio/video.
- **Moderazione Globale Commenti:** solo l'Admin può eliminare qualsiasi commento nel sistema.
- **Cancellazione Condizionale Social:** un utente normale può eliminare i commenti di altri utenti, ma solo sui brani che ha caricato lui stesso.

3.3 Diagramma dei Casi d'Uso

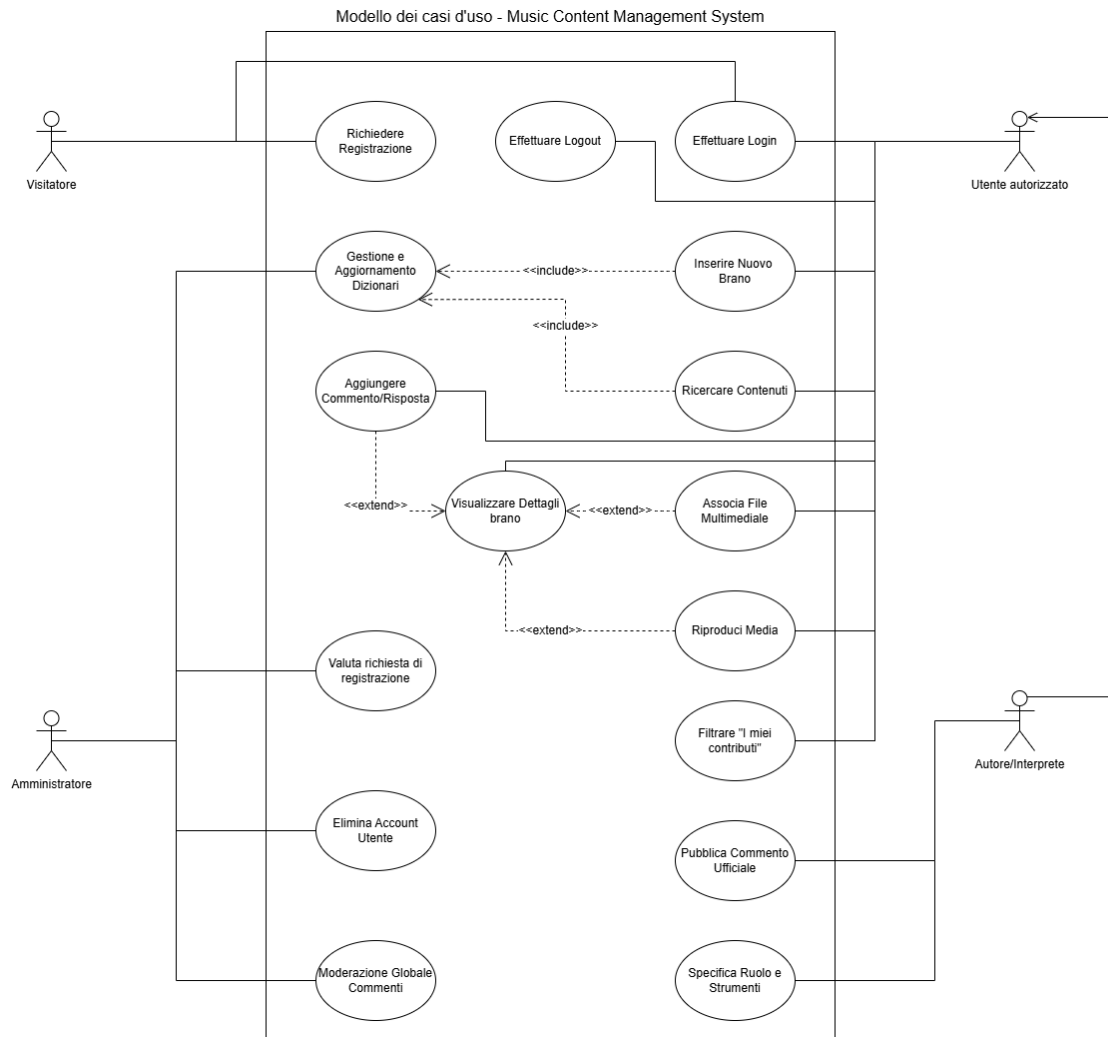


Figura 3.1: Diagramma dei Casi d'Uso con attori, confini del sistema e relazioni di inclusione/estensione.

3.4 Giustificazione delle relazioni UML

3.4.1 Relazioni di Inclusione (<<include>>)

La relazione di **include** indica una dipendenza obbligatoria: il caso d'uso base non può completarsi senza quello incluso.

Da *Inserire Nuovo Brano* verso *Gestione Dizionari*: il sistema non accetta generi musicali a testo libero. Prima di salvare un brano, controlla obbligatoriamente che il genere esista nel dizionario; se non c'è, l'inserimento si blocca. Abbiamo scelto **include** perché non è un comportamento opzionale: senza questa verifica il database si frammenterebbe inevitabilmente. Da *Ricerca Contenuti* verso *Gestione Dizionari*: le ricerche per genere e per autore hanno senso solo grazie al vocabolario controllato gestito dai dizionari, che normalizza i termini al momento dell'inserimento ed evita che varianti di battitura rendano i brani irreperibili in fase di interrogazione.

3.4.2 Relazioni di Estensione («extend»)

La relazione di **extend** modella comportamenti opzionali che si attivano solo in certe condizioni. Da *Aggiungi Commento*, *Associa File* e *Riproduci Media* verso *Visualizzare Dettaglio Brano*: dalla Dashboard principale si vede solo la tabella dei brani. Per commentare o riprodurre un file bisogna prima aprire la scheda di dettaglio, dove questi pulsanti appaiono e diventano attivi. Abbiamo usato **extend** perché sono comportamenti condizionati all'apertura di una specifica vista, non sempre disponibili.

3.5 Schede di Specifica dei Casi d'Uso

3.5.1 UC-01: Inserire Nuovo Brano

Tabella 3.1: Scheda di specifica per UC-01.

Elemento	Descrizione
ID e Nome	UC-01: Inserire Nuovo Brano
Attori	Utente Autorizzato (Primario), Autore/Interprete
Pre-condizioni	L'utente è loggato, la sessione è attiva nel SessionManager e ha aperto la finestra "Nuovo Brano".
Scenario Principale	1. L'utente compila i campi Titolo, Autore Principale e Anno di Composizione. 2. Il sistema verifica che il genere e l'autore principale inseriti esistano nei rispettivi dizionari controllati (Generi e Autori; caso d'uso incluso). 3. L'utente preme "Salva nel Database". 4. Il sistema converte l'anno in numero intero e controlla che non ci siano duplicati nel catalogo. 5. Il brano viene scritto nel file JSON e la UI mostra un messaggio di conferma.
Scenari Alternativi	3a. Genere o autore non nei dizionari: il sistema blocca il salvataggio e avvisa l'utente che il genere o l'autore inserito non è valido. 4a. Anno non numerico: viene intercettata la <code>NumberFormatException</code> e la scrittura su disco non avviene.
Post-condizioni	Il brano è inserito nel file JSON ed è subito visibile nella TableView della Dashboard.

3.5.2 UC-02: Ricercare Contenuti

Tabella 3.2: Scheda di specifica per UC-02.

Elemento	Descrizione
ID e Nome	UC-02: Ricercare Contenuti
Attori	Utente Autorizzato, Autore/Interprete, Amministratore
Pre-condizioni	La Dashboard è aperta e nel catalogo c'è almeno un brano.

Elemento	Descrizione
Scenario Principale	1. L'utente compila uno o più dei quattro campi di filtro indipendenti (<code>filterTitolo</code>, <code>filterAutore</code>, <code>filterGenere</code>, <code>filterEsecutore</code>). 2. L'utente preme "Applica Filtri". 3. Il sistema normalizza in minuscolo ciascun criterio compilato (per ignorare maiuscole/minuscole) e avvia una pipeline Stream API. 4. Lo stream applica in AND i filtri valorizzati su titolo, autore e genere; per l'esecutore usa un <code>flatMap</code> che scende nei segmenti temporali dei documenti associati (<i>deep scanning</i>). I campi lasciati vuoti non vincolano la ricerca. 5. I risultati vengono passati alla TableView che si aggiorna.
Scenari Alternativi	4a. Nessun risultato: viene sollevata l'eccezione custom <code>BranoNonTrovatoException</code> , la tabella viene svuotata e appare un messaggio di errore.
Post-condizioni	La TableView mostra solo i brani che corrispondono alla ricerca; la lista completa in RAM resta intatta.

3.5.3 UC-03: Aggiungi Commento/Risposta

Tabella 3.3: Scheda di specifica per UC-03.

Elemento	Descrizione
ID e Nome	UC-03: Aggiungi Commento/Risposta
Attori	Utente Autorizzato
Scenario Principale	1. L'utente scrive il suo commento in <code>nuovoCommentoField</code>. 2. Il sistema recupera l'email dell'autore dal <code>SessionManager</code>. 3. L'utente preme "Invia". 4. Viene creato un oggetto <code>Commento</code> con testo, data corrente e autore. 5. Se l'utente è un Autore/Interprete del brano, può marcare il commento come ufficiale tramite checkbox. 6. Il commento viene agganciato direttamente al brano o annidato come risposta a un commento esistente (Pattern Composite). 7. I dati vengono scritti su disco e la TreeView visiva si aggiorna.
Scenari Alternativi	1a. Campo vuoto: se il testo è composto da soli spazi, il sistema lo intercetta con <code>trim().isEmpty()</code> e non fa nulla.
Post-condizioni	La struttura dei commenti viene aggiornata su file e la UI mostra la gerarchia con i rientri corretti.

3.5.4 UC-04: Gestione e Aggiornamento Dizionari

Tabella 3.4: Scheda di specifica per UC-04.

Elemento	Descrizione
ID e Nome	UC-04: Gestione e Aggiornamento Dizionari

Elemento	Descrizione
Attori	Amministratore di Sistema (esclusivo)
Pre-condizioni	L'Admin è loggato e ha aperto il pannello <code>gestione_dizionari.fxml</code> .
Scenario Principale	1. L'Admin seleziona dalla ChoiceBox quale dizionario vuole modificare (Generi, Strumenti, Titoli, Autori). 2. La ListView mostra le voci attualmente presenti nel file <code>dizionari.json</code>. 3. L'Admin digita il nuovo termine in nuovaVoceField e clicca "Aggiungi". 4. Il sistema controlla che la voce non esista già nel dizionario. 5. La nuova stringa viene inserita nella lista del dizionario selezionato. 6. Il GestoreJSON riscrive il file su disco.
Scenari Alternativi	4a. Termine già presente: viene stampato un avviso e l'inserimento non avviene.
Post-condizioni	Il dizionario è aggiornato su disco e la nuova voce è subito disponibile per ricerche e inserimenti.

3.5.5 UC-05: Richiedere Registrazione

Tabella 3.5: Scheda di specifica per UC-05.

Elemento	Descrizione
ID e Nome	UC-05: Richiedere Registrazione
Attori	Visitatore (Guest)
Pre-condizioni	Il sistema è sulla schermata di Login e il visitatore non ha ancora un account.
Scenario Principale	1. Il Visitatore clicca su "Richiedi Registrazione". 2. Il sistema carica il form di iscrizione. 3. Il Visitatore compila i campi e sceglie il ruolo (Utente Standard o Autore/Interprete). 4. Se viene scelto il ruolo Autore/Interprete, compare la textarea (facoltativa) per la biografia. 5. Il Visitatore clicca su "Invia Richiesta". 6. Il sistema verifica che i campi obbligatori (nome, email, password) non siano vuoti, controlla l'unicità dell'email con le Stream API, genera un UUID come ID e crea l'istanza con <code>attivo = false</code>. 7. Il record viene salvato in <code>utenti.json</code> in attesa di approvazione.
Scenari Alternativi	6a. Email già registrata: la scrittura viene bloccata e la UI mostra "Email già registrata".
Post-condizioni	I dati del Visitatore sono salvati in stato disattivato e visibili nel pannello admin.

3.5.6 UC-06: Valutare Richieste

Tabella 3.6: Scheda di specifica per UC-06.

Elemento	Descrizione
ID e Nome	UC-06: Valutare Richieste
Attori	Amministratore di Sistema (esclusivo)
Pre-condizioni	L'Admin è nel pannello <code>gestione_utenti.fxml</code> e ci sono account con <code>attivo = false</code> nel database.
Scenario Principale	1. L'Admin vede la tabella con le richieste in sospenso. 2. Seleziona un utente dalla lista. 3. Clicca su "Approva Utente". 4. Il sistema recupera l'oggetto selezionato e imposta il flag <code>attivo = true</code>. 5. Il <code>GestoreJSON</code> riscrive la lista aggiornata in <code>utenti.json</code>. 6. La tabella si aggiorna per riflettere il nuovo stato.
Scenari Alternativi	3a. Rifiuto: l'Admin clicca su "Elimina Account". Appare un alert di conferma per evitare eliminazioni accidentali; se confermato, l'utente viene rimosso dal file JSON.
Post-condizioni	Se approvato, l'utente può ora fare login. Se eliminato, il record non esiste più.

Capitolo 4

Modellazione Concettuale e Progettazione Statica

Questo capitolo descrive l'architettura statica di MusicManager attraverso il Class Diagram UML. A differenza delle bozze iniziali, il diagramma che presentiamo qui rispecchia fedelmente il codice Java effettivo: abbiamo inserito modificatori di visibilità, molteplicità, e tutti i design pattern che abbiamo applicato.

4.0.1 Come siamo arrivati al diagramma definitivo

Abbiamo lavorato su due livelli di astrazione distinti.

Vista Concettuale (Domain Model)

In un primo momento ci siamo concentrati solo sulle entità del dominio e sui loro rapporti, senza pensare a Java, ai tipi di dato o alle strutture dati. Il nucleo del catalogo ruota attorno a **Brano**, che aggrega in composizione i propri **DocumentoDigitale** (con le quattro specializzazioni **TracciaAudio**, **TracciaVideo**, **Spartito** e **CollegamentoWeb**) e i propri **Commento**. Ogni **DocumentoDigitale** può a sua volta essere suddiviso in più **SegmentoEsecuzione** (molteplicità 0..*), che mappano i singoli brani all'interno della registrazione di un concerto. La struttura dei feedback è ricorsiva: un **Commento** può contenere altri **Commento** come risposte, mentre la sua specializzazione **NotaTemporale** àncora il feedback a un intervallo di tempo del media. I termini ammessi per i metadati sono governati dalla classe **Dizionario**. La gerarchia degli utenti parte dalla radice astratta **Utente** e si dirama nelle classi concrete: **Visitatore** e **Amministratore** ereditano direttamente da **Utente**, mentre **AutoreInterprete** è una specializzazione di **UtenteAutorizzato** (ereditarietà su tre livelli). Accanto al catalogo, **Playlist** modella una raccolta ordinata di brani ed è inclusa per dimostrare il pattern Iterator: è un'entità concettuale del dominio, non collegata a una specifica vista dell'interfaccia. In questa fase le classi non hanno getter/setter, tipi primitivi o collezioni specifiche di Java: è solo un modello logico.

Vista del Software (Implementation Model)

Il diagramma definitivo (Figura 4.1) è l'evoluzione concreta del modello, dove l'astrazione ha lasciato spazio alle scelte implementative reali. Tutti gli attributi sono **private** (-), i metodi pubblici +: nessuno può toccare lo stato interno di un oggetto senza passare dai suoi metodi. Le collezioni interne (**documentiAssociati**, **commenti**) vengono inizializzate come **new ArrayList<>()** direttamente nei costruttori, eliminando alla radice il rischio di **NullPointerException** — una precauzione nata dopo qualche bug spiacevole nelle prime sessioni di test. Abbiamo poi integrato il design pattern Singleton per **SessionManager**, il pattern Composite per la struttura ricorsiva dei commenti, e

l'Iterator con inner class privata per Playlist. La serializzazione con Gson è affidata interamente alla classe `GestoreJSON`. Infine, invece di far puntare `Brano` direttamente a `Utente` (il che causerebbe riferimenti circolari durante la serializzazione JSON e un bel `StackOverflowError`), usiamo stringhe come `proprietarioEmail` e `autoreEmail`: la stessa idea delle foreign key in un database relazionale.

4.1 Il Class Diagram

Nel diagramma finale non compaiono le due classi controller della prima iterazione (*GestoreCatalogo* e *GestoreUtenze*): la logica applicativa è stata interamente assorbita dai controller JavaFX dell’architettura MVC. Le due classi sono rimaste nel codice sorgente (package `com.musicmanager.controller`) come residuo della progettazione iniziale, ma non sono più collegate al flusso attivo dell’applicazione: nessun controller di vista le istanzia e possono essere considerate codice legacy.

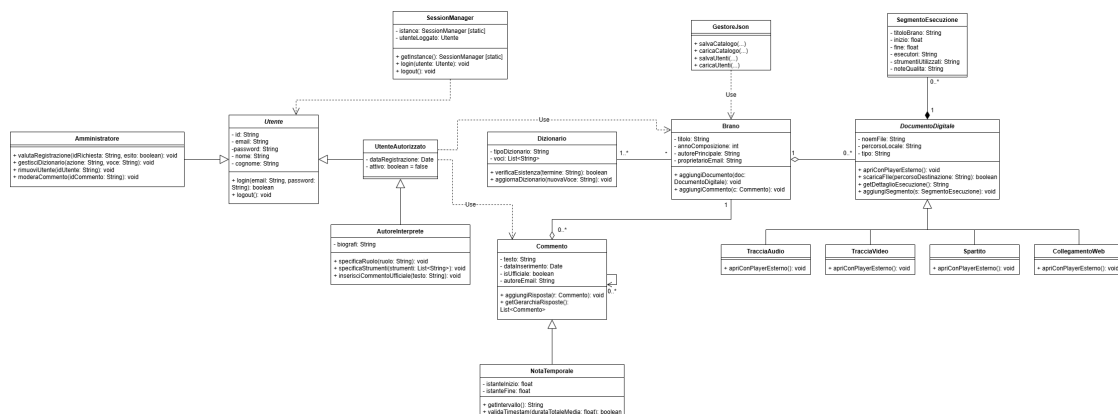


Figura 4.1: Class Diagram definitivo del sistema: classi di model, controller e persistence con visibilità e molteplicità.

4.2 Analisi delle classi per package

Il progetto è organizzato in package gerarchici. Tutti gli attributi sono **private** (-) e i metodi **public** (+): questo impedisce modifiche dirette dall'esterno e obbliga a usare getter/setter o metodi di business espliciti.

4.2.1 Package utenti (com.musicmanager.model.users)

Classe Astratta Utente

È la radice della gerarchia degli account. È **abstract** (in corsivo nell'UML) quindi non si può istanziare direttamente. Ha i campi anagrafici base (**id**, **email**, **password**, **nome**, **cognome**) e i metodi **login()** e **logout()**.

Classe Visitatore

Rappresenta l'utente non autenticato. Eredita da `Utente` e non aggiunge campi (per tenere l'impronta di memoria minima); il metodo `richiediRegistrazione()` ne modella, a livello concettuale, l'unica azione consentita. Nell'implementazione corrente la richiesta di iscrizione è gestita direttamente dal `RegistrazioneController`, che istanzia subito l'`UtenteAutorizzato` (o l'`AutoreInterprete`) con `attivo = false`: la

classe **Visitatore** resta quindi l'attore di dominio del caso d'uso, senza essere istanziata nel flusso grafico.

Classe UtenteAutorizzato

L'utente base approvato dall'admin. Aggiunge due campi importanti: **dataRegistrazione** (`java.util.Date`), che registra quando si è iscritto, e **attivo** (`boolean`), inizializzato a `false`. Il **LoginController** controlla questo flag: anche con password corretta, se è `false` il login viene bloccato con il messaggio "Account in attesa di approvazione".

Classe AutoreInterprete

Specializzazione di terzo livello: eredita da **UtenteAutorizzato** (non da **Utente** direttamente). Questo è importante perché così eredita nativamente il flag **attivo** e la gestione dell'approvazione. Aggiunge il campo **biografia** (`String`) e metodi come **specificaRuolo()**, **specificaStrumenti()** e **inserisciCommentoUfficiale()**.

Classe Amministratore

Eredita da **Utente** e aggiunge i massimi privilegi: **valutaRegistrazione()**, **gestisciDizionario()**, **rimuoviUtente()** e **moderaCommento()**.

4.2.2 Package catalogo (`com.musicmanager.model.catalog`)

Classe Brano

È il centro di tutto il sistema. Contiene titolo, autore principale, genere, anno di composizione e **proprietarioEmail**. Le due liste interne (**documentiAssociati** e **commenti**) vengono inizializzate come `new ArrayList<>()` nel costruttore: una scelta difensiva che elimina alla radice il rischio di `NullPointerException`.

Classe Dizionario

Gestisce il controllo tassonomico. Ha un nome (**tipoDizionario**, es. "Generi") e una lista di stringhe **voci**. Il metodo **verificaEsistenza(termine: String): boolean** usa **contains()** per validare l'input.

Classe Astratta DocumentoDigitale

Base per tutti i file associabili. Ha il **nomeFile**, il **percorsoLocale** e il **tipo**. È in relazione di composizione (rombo pieno) con **SegmentoEsecuzione**: se si elimina il documento, tutti i suoi segmenti vengono distrutti insieme. Dichiarare il metodo astratto **apriConPlayerEsterno()**.

Classi TracciaAudio, TracciaVideo, Spartito, CollegamentoWeb

Implementano concretamente **apriConPlayerEsterno()** usando `java.awt.Desktop`: la JVM chiede al sistema operativo di aprire il file con il programma predefinito. Così l'app Java rimane leggera e non deve gestire decodifica multimediale.

Classe SegmentoEsecuzione

Modella la suddivisione dei video di concerti. Ha i marker **inizio** e **fine** in `float` (per gestire i millisecondi), il titolo del brano eseguito, gli esecutori e gli strumenti usati. Il campo **noteQualita** serve per annotazioni sulla performance.

4.2.3 Package feedback (`com.musicmanager.model.feedback`)

Classe Commento

L'entità base delle interazioni sociali. Contiene il testo, la data, l'`autoreEmail` e il flag `isUfficiale` per distinguere i commenti degli autori degli artisti.

Classe NotaTemporale

Eredita da `Commento` e aggiunge `istanteInizio` e `istanteFine` in `float`. Il metodo `validaTimestamp(durataMedia: float): boolean` controlla che i valori abbiano senso (inizio non negativo, inizio prima della fine, fine non oltre la durata del media) prima di permettere il salvataggio.

4.3 Design Pattern strutturali

4.3.1 Pattern Composite — commenti annidati

I requisiti chiedevano commenti e risposte “senza limiti di profondità”. Il pattern Composite era la soluzione più pulita: la classe `Commento` fa sia da foglia che da contenitore, grazie a un attributo riflessivo:

```
private List<Commento> risposte = new ArrayList<>();
```

Ogni commento può contenere altri commenti, che a loro volta ne contengono altri, generando un albero di profondità arbitraria. Questo si integra perfettamente con la `TreeView` di `JavaFX`, che itera ricorsivamente la gerarchia tramite il metodo `getGerarchiaRisposte()` per costruire i rientri visivi nell'interfaccia.

4.3.2 Pattern Iterator — scorrere la Playlist

La classe `Playlist` implementa `Iterable<Brano>` e usa un'inner class privata come iteratore:

```
public class Playlist implements Iterable<Brano> {
    private List<Brano> collezioneBrani = new ArrayList<>();

    @Override
    public Iterator<Brano> iterator() {
        return new PlaylistIterator();
    }

    private class PlaylistIterator implements Iterator<Brano>
    {
        private int cursor = 0;

        @Override
        public boolean hasNext() {
            return cursor < collezioneBrani.size();
        }

        @Override
        public Brano next() {
            return collezioneBrani.get(cursor++);
        }
    }
}
```

```
    }  
}
```

Questa soluzione ha due vantaggi pratici. L'inner class privata `PlaylistIterator` può accedere direttamente a `collezioneBrani` senza che serva un getter pubblico che esponga la lista all'esterno. I controller grafici possono scorrere la playlist con un `for-each` normale, senza sapere nulla di come è implementata internamente.

4.3.3 Pattern Singleton — gestione della sessione

`SessionManager` usa il Singleton classico con lazy initialization: l'istanza viene creata la prima volta che si chiama `getInstance()` e poi viene restituito sempre lo stesso oggetto. Il costruttore privato impedisce istanziazioni accidentali da altri moduli. In questo modo tutti i controller dell'applicazione vedono sempre lo stesso stato di autenticazione.

4.4 Riferimenti circolari e chiavi logiche

Sul Class Diagram ci sono alcune frecce tratteggiate che collegano il package utenti e `GestoreJson` al catalogo. Queste rappresentano dipendenze, non associazioni forti, e la scelta è legata alla serializzazione JSON.

4.4.1 Il problema

In teoria, `Brano` dovrebbe avere un riferimento diretto all'oggetto `Utente` che lo ha caricato, e `Commento` dovrebbe puntare all'utente autore. Ma Gson, quando serializza, seguirebbe i riferimenti: il brano chiama l'utente, l'utente ha la lista dei suoi brani, che chiamano i commenti, che ripuntano all'utente, e così via all'infinito fino a `StackOverflowError`.

4.4.2 La soluzione: chiavi logiche

Abbiamo risolto con lo stesso concetto delle foreign key dei database relazionali: `Brano` memorizza `proprietarioEmail` come stringa, `Commento` memorizza `autoreEmail` come stringa. I due moduli non sono accoppiati direttamente e si parlano solo attraverso i controller, che fanno da mediatori. Le frecce tratteggiate nel diagramma rappresentano esattamente queste dipendenze nei metodi dei controller.

Capitolo 5

Modellazione Dinamica e Comportamentale

Dopo aver descritto la struttura statica del sistema, questo capitolo mostra come si comporta a runtime. Usiamo due tipi di diagrammi: i **Sequence Diagram** per descrivere lo scambio di messaggi tra gli oggetti nel tempo, e gli **Activity Diagram** per mostrare il flusso procedurale delle operazioni principali.

5.1 Diagrammi di Sequenza

I Sequence Diagram mostrano gli oggetti in orizzontale e il tempo che scorre verso il basso. Le barre di attivazione indicano quando un oggetto è “attivo”, cioè sta eseguendo qualcosa.

5.1.1 SD-01: Inserimento di un nuovo brano

Il diagramma **SD-01** (Figura 5.1) descrive cosa succede quando un utente preme il pulsante “Salva” nel form di inserimento. Mappa il metodo `handleSalva(ActionEvent event)` di `AggiungiBranoController`.

Come si svolge la sequenza

Il flusso si divide in tre fasi principali:

1. Parsing dell’input

L’utente preme il pulsante e viene invocato `handleSalva(event)` sul Controller. Subito, il Controller fa un’auto-chiamata per fare il parsing dell’anno con `Integer.parseInt(annoStr)`: primo controllo difensivo contro input non numerici.

2. Recupero del contesto

Il Controller carica il catalogo da disco invocando `caricaCatalogo("catalogo.json")` su `gestore:GestoreJSON`, che restituisce la lista dei brani. Poi interroga il `SessionManager` con `getInstance()` e `getUtenteLoggato().getEmail()` per recuperare l’email dell’utente loggato (serve come “proprietario” del brano).

3. Blocco condizionale (alt)

Se il titolo non è duplicato e i campi sono completi, il Controller crea il brano con «create» `new Brano(t, anno, a, email, g)`, lo aggiunge al catalogo e chiama `salvaCatalogo()` per persistere tutto. La UI mostra “Salvato!”. In presenza di un errore o di un duplicato, il blocco `alt` va nel ramo alternativo, nessuna memoria viene allocata, e appare un messaggio di errore.

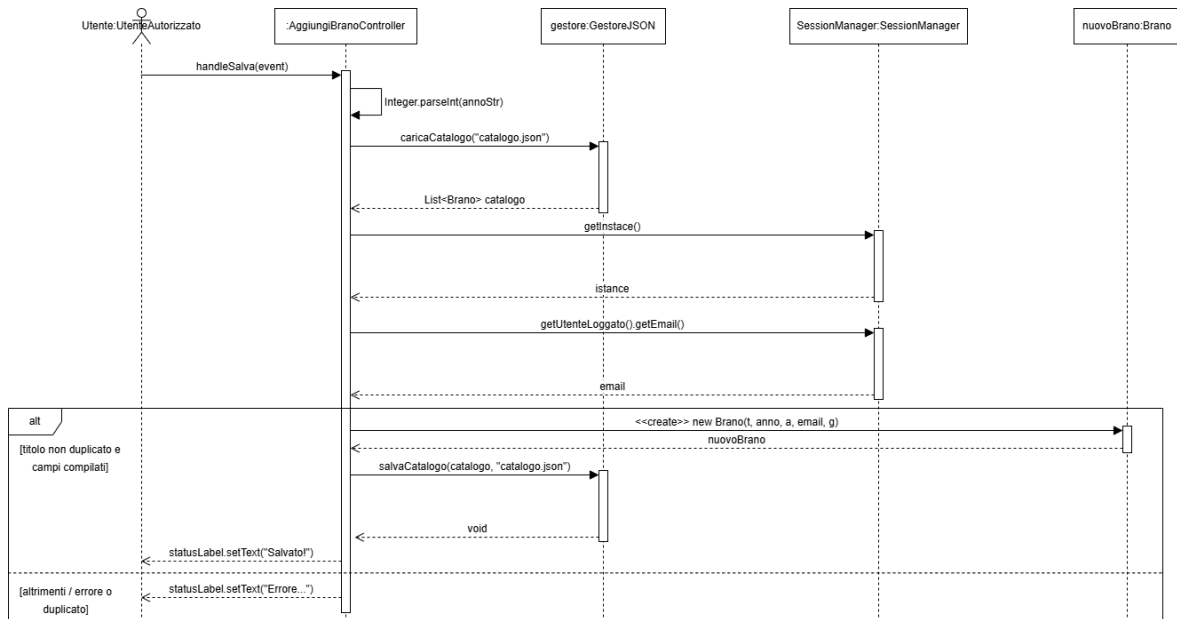


Figura 5.1: SD-01: Sequenza per l'inserimento e validazione di un nuovo brano.

5.1.2 SD-02: Inserimento di un commento o risposta

Il diagramma **SD-02** (Figura 5.2) mostra il pattern Composite in azione: come il sistema distingue tra un commento radice e una risposta annidata a runtime. Corrisponde al metodo `handleInviaCommento()` di `DettaglioController`.

Come si svolge la sequenza

1. Sanitizzazione e sessione

L'utente preme "Invia" e il Controller legge il testo con `getText().trim()`, poi recupera l'email dell'autore dal Singleton (`getInstance() + getUtenteLoggato().getEmail()`).

2. Creazione del nodo e bivio (alt)

Viene creato un nuovo oggetto `Commento` con `<<create>> new Commento(...)`. Poi il blocco `alt` decide dove agganciarlo: se c'è una selezione nella `TreeView`, il Controller chiama `aggiungiRisposta(nuovoCommento)` sul commento padre selezionato, che lo aggiunge alla propria lista interna (è qui che si vede concretamente il pattern Composite); se non c'è selezione, viene chiamato direttamente `branoCorrente.aggiungiCommento(nuovoCommento)`.

3. Persistenza e aggiornamento UI

Il Controller carica il catalogo, chiama `salvaCatalogo()` per scrivere su disco la struttura ad albero aggiornata, poi pulisce il campo testo e aggiorna la `TreeView` visiva.

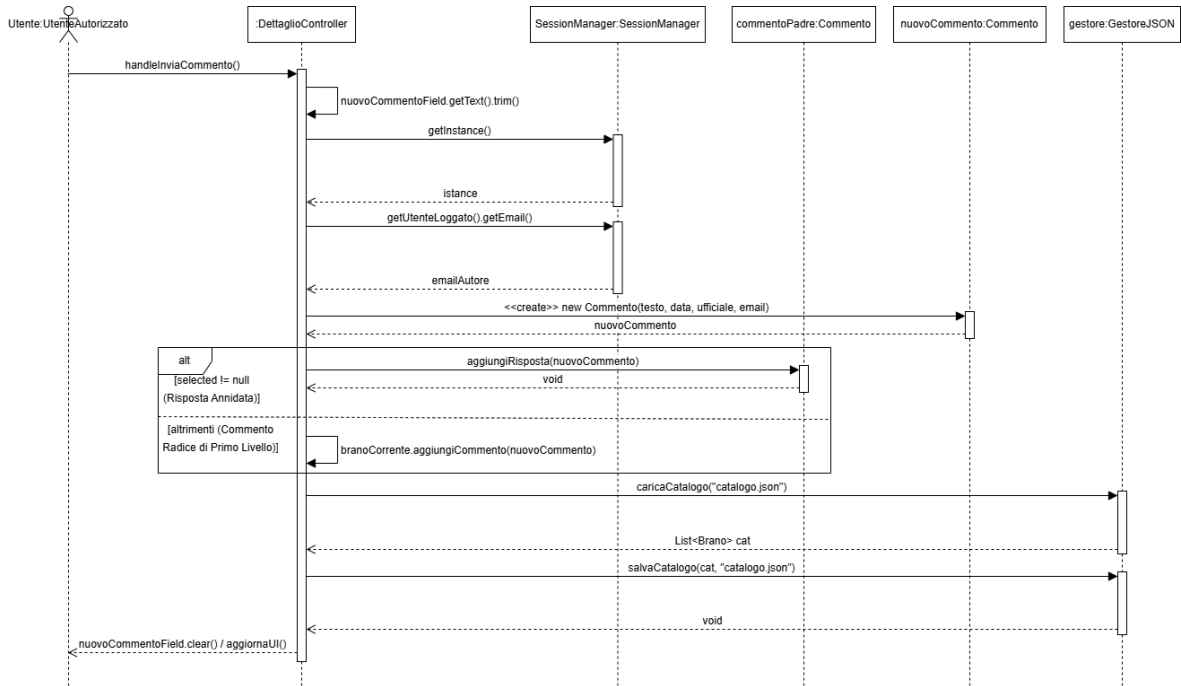


Figura 5.2: SD-02: Sequenza per l'inserimento ricorsivo di commenti e risposte (Pattern Composite).

5.1.3 SD-03: Ricerca avanzata con eccezione custom

Il diagramma **SD-03** (Figura 5.3) mostra come funziona il motore di ricerca, combinando le Stream API (rappresentate come un loop nello schema) e la gestione di un'eccezione personalizzata. Corrisponde a `handleRicerca(ActionEvent event)` in `DashboardController`.

Come si svolge la sequenza

1. Acquisizione e normalizzazione

L'utente preme "Applica Filtri" e il Controller legge i quattro campi di filtro, normalizzando ciascuno con `getText().trim().toLowerCase()`. Poi richiede lo stream sulla lista master con `stream()`.

2. Blocco loop — il filtraggio

Per ogni brano nello stream vengono applicati in AND i quattro predicati corrispondenti ai criteri valorizzati: titolo su `getTitolo()`, autore su `getAutorePrincipale()`, genere su `getGenere()` ed esecutore sui segmenti dei documenti associati (tramite `flatMap`: questo è il cosiddetto deep scanning). Un criterio vuoto rende sempre vero il proprio predicato; ogni brano restituisce così `true` o `false` al Controller.

3. Blocco alt — risultati o eccezione

Se la lista è vuota, viene creata e lanciata una `BrancoNonTrovatoException`, catturata nel blocco `catch`: il messaggio di errore appare in `errorSearchLabel` e la tabella viene svuotata. Se ci sono risultati, la tabella viene aggiornata con `tabellaBrani.setItems(filtrati)`.

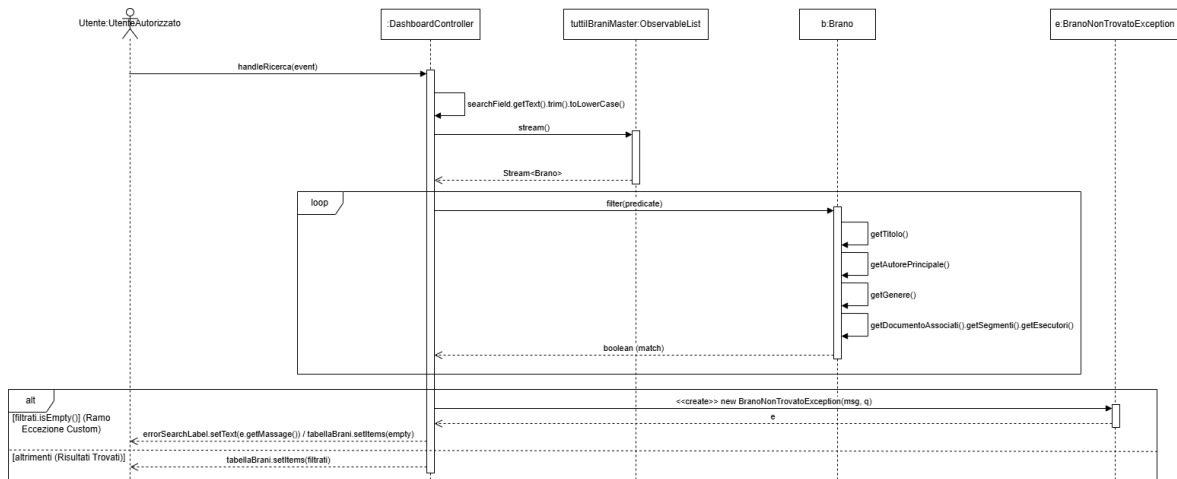


Figura 5.3: SD-03: Sequenza per la ricerca funzionale avanzata con eccezione custom.

5.1.4 SD-04: Approvazione di una richiesta di registrazione

Il diagramma **SD-04** (Figura 5.4) descrive il workflow di approvazione da parte dell'admin, mostrando come un flag booleano viene modificato in RAM e poi sincronizzato su disco. Corrisponde a `handleApprova()` in `GestioneUtentiController`.

Come si svolge la sequenza

1. Selezione del record

L'Admin preme "Approva Utente" e il Controller identifica l'utente selezionato con `tabellaUtenti.getSelectionModel().getSelectedItem()`.

2. Mutazione dello stato in RAM

Viene chiamato `setAttivo(true)` sull'istanza selezionata: il flag booleano privato cambia da `false` a `true`.

3. Sincronizzazione su disco

Il Controller chiama `salvaUtenti(listaUtenti, "utenti.json")` sul `GestoreJSON`, che riscrive il file con Gson.

4. Refresh visivo

Viene chiamato `tabellaUtenti.refresh()` per invalidare la cache grafica e ridisegnare la riga aggiornata senza riavviare la schermata.

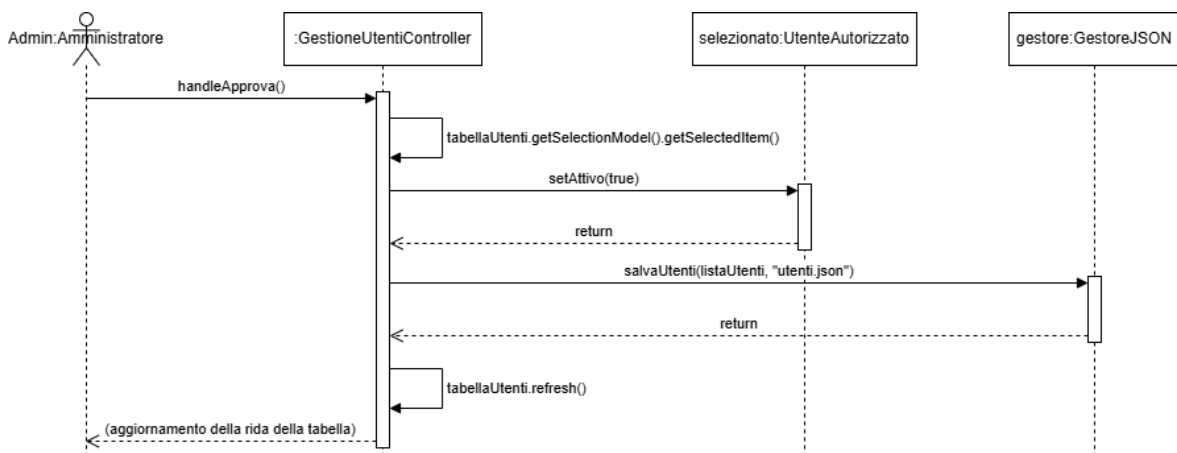


Figura 5.4: SD-04: Sequenza per l'approvazione di una richiesta di registrazione.

5.2 Diagrammi delle Attività

I Diagrammi delle Attività descrivono il flusso di controllo procedurale: come si muove il “gettone” attraverso nodi di azione, nodi decisionali e nodi di convergenza. Sono utili per visualizzare i casi limite e i cicli di rientro in caso di errore.

5.2.1 AD-00: Ciclo di vita globale del sistema

Il diagramma **AD-00** (Figura 5.5) è la mappa complessiva dell'applicazione, dal bootstrap iniziale alla chiusura.

Il percorso principale

1. **Bootstrap** — la JVM inizializza JavaFX e lo Scene Graph. Il gettone passa per un Merge Node che raccoglie anche i rientri dai logout successivi.
2. **Schermata di login** — qui il sistema blocca tutti gli utenti non autenticati. Dal form di login si può andare verso la registrazione (i dati vengono salvati con `attivo=false` e si torna alla schermata di login) oppure verso la verifica delle credenziali.
3. **Controllo RBAC** — dopo il login il sistema controlla con `instanceof` se l'utente è un `Amministratore` o un `UtenteAutorizzato` normale e carica la Dashboard con i controlli appropriati. I due flussi si ricongiungono subito dopo.
4. **Loop operativo** — l'utente usa l'applicazione finché non sceglie di fare logout (`SessionManager.getInstance().logout()`, pulizia della RAM, rientro alla schermata di login) o di chiudere la finestra (il gettone raggiunge il nodo finale, il processo termina).

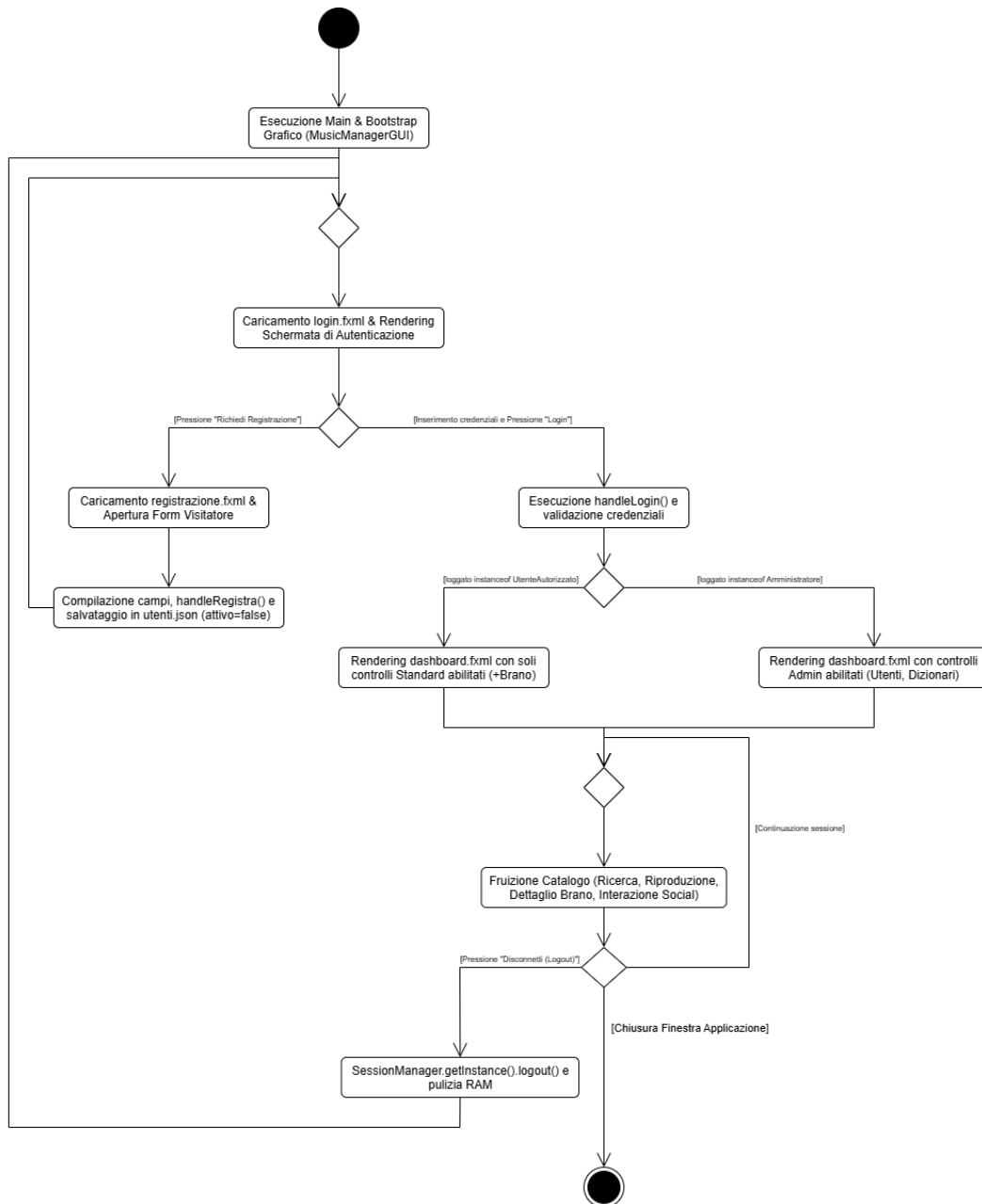


Figura 5.5: AD-00: Ciclo di vita globale dell'applicazione.

5.2.2 AD-01: Flusso di login

Il diagramma **AD-01** (Figura 5.6) descrive nel dettaglio il metodo `handleLogin()` di `LoginController`, mostrando come funziona la barriera di sicurezza.

Il percorso

1. **Lettura dei campi e bypass admin** — se email e password corrispondono all'account admin hardcoded (`admin@musicmanager.it` / `admin123`), si va direttamente alla Dashboard senza leggere il JSON.
2. **Ciclo di ricerca** — per il login normale, il sistema carica `utenti.json` e itera sugli utenti. Se le credenziali non corrispondono si passa al prossimo utente; se si finisce la lista senza match appare l'errore "Credenziali errate" e si torna alla lettura iniziale.

3. **Controllo del flag attivo** — quando le credenziali sono giuste, si verifica il flag: se **false**, appare il messaggio “Account in attesa di approvazione” e si rientra alla schermata di login; se **true**, il login viene registrato nel **SessionManager** e si apre la Dashboard.

Password corretta non basta: serve anche il flag **attivo**. Questa doppia condizione è la barriera di sicurezza che impedisce agli utenti appena registrati di entrare prima che l’admin li abbia approvati.

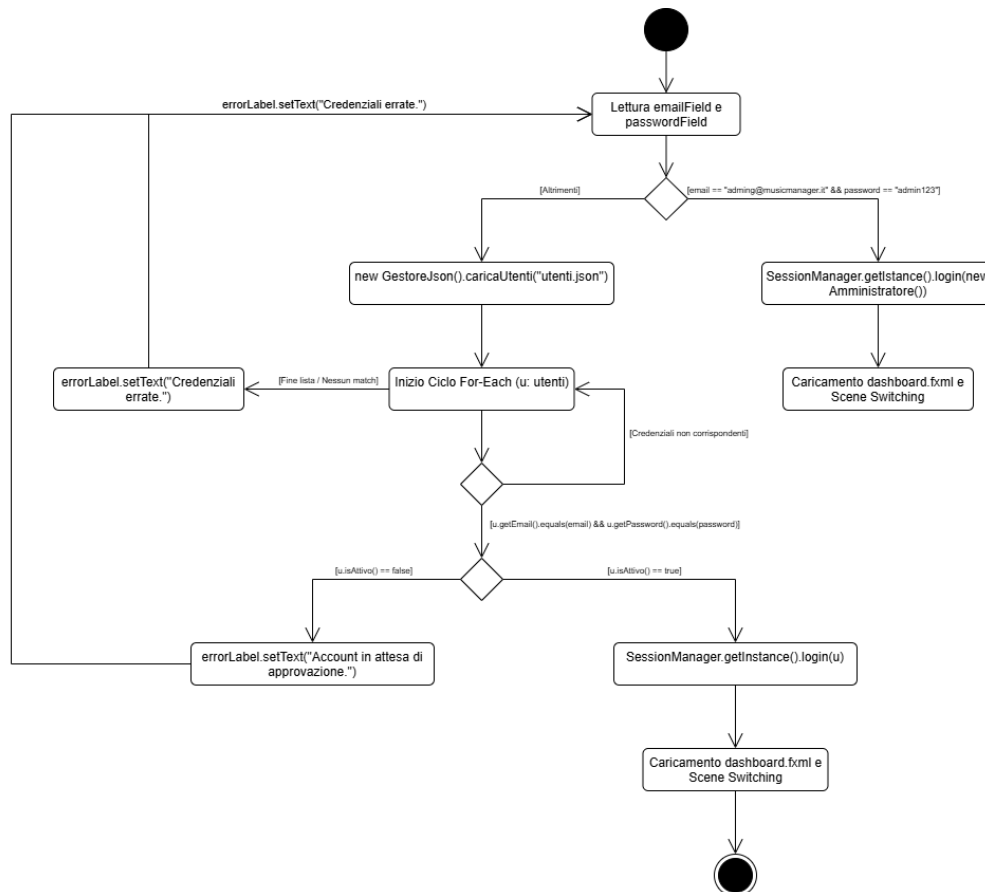


Figura 5.6: AD-01: Flusso di autenticazione e controllo dello stato dell’account.

5.2.3 AD-02: Pipeline di registrazione

Il diagramma **AD-02** (Figura 5.7) mostra il metodo `handleRegistra()` di `RegistrazioneController`, che combina la mutazione dinamica della UI con una pipeline di validazione a tre livelli.

Il percorso

1. **Adattamento del layout** — se l’utente sceglie “Autore/Interprete” dalla ChoiceBox, la textarea per la biografia diventa visibile e gestita (`setVisible(true)` + `setManaged(true)`); altrimenti viene nascosta. Servono entrambe le proprietà: solo `setVisible(false)` lascia lo spazio vuoto nel layout.
2. **Primo scudo: campi obbligatori** — se nome, email o password sono vuoti, appare l’errore e si rientra al punto di partenza.

3. **Secondo scudo: unicità dell'email** — se l'email è già nel database, appare “Email già registrata” e non si va avanti.
4. **Istanziamento polimorfico** — superati i controlli, viene generato un UUID come ID e viene creata l'istanza giusta (**UtenteAutorizzato** o **AutoreInterprete** con la biografia). Il flag **attivo** viene impostato a **false** e il record viene scritto su disco.

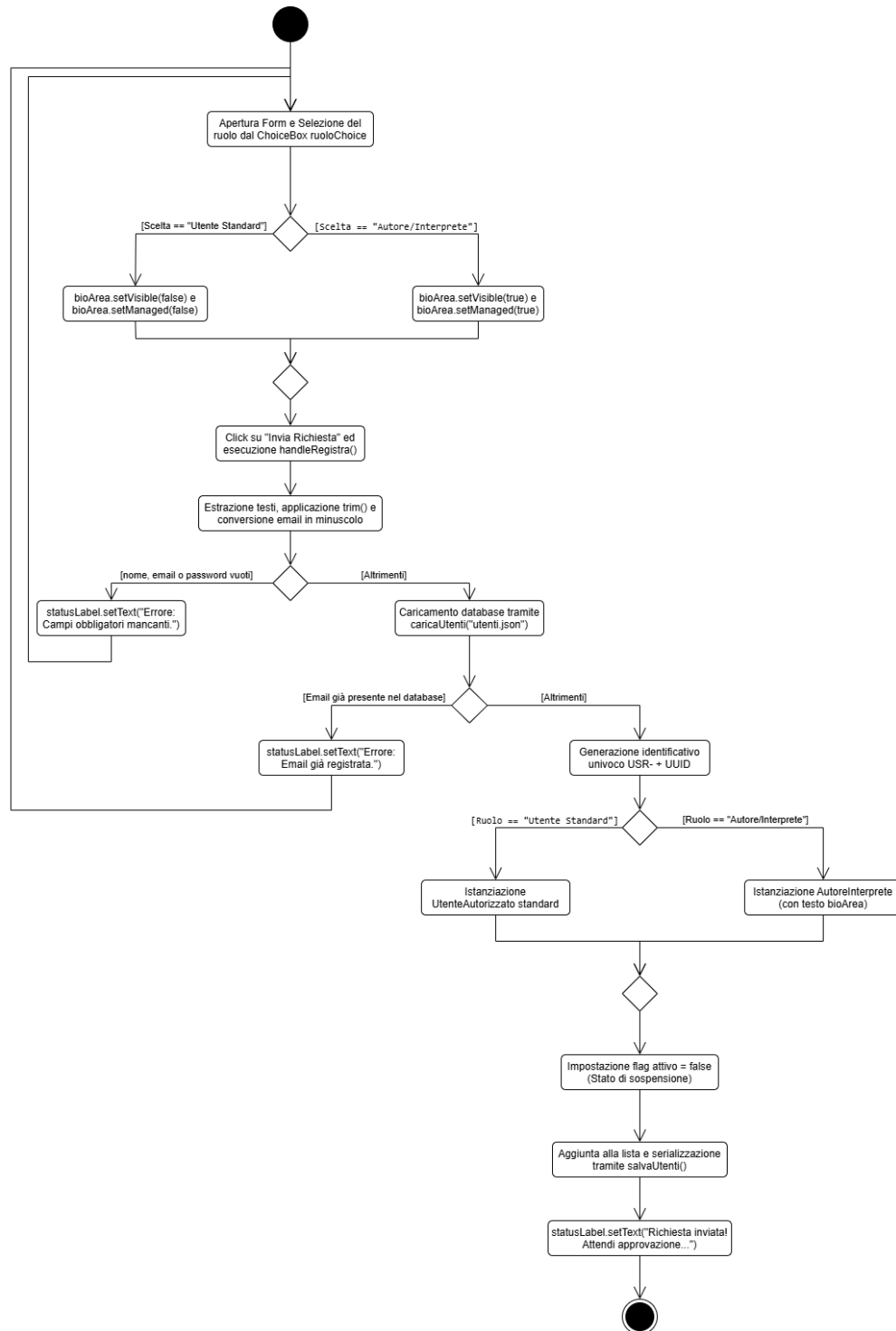


Figura 5.7: AD-02: Pipeline di registrazione con validazione e istanziamento polimorfico.

5.2.4 AD-03: Salvataggio e modifica di un brano

Il diagramma **AD-03** (Figura 5.8) descrive `handleSalva()` di `AggiungiBranoController`. Lo stesso form e lo stesso metodo gestiscono sia l’inserimento di un brano nuovo che la modifica di uno esistente: il comportamento cambia in base al valore di un puntatore (`branoInModifica`).

Il percorso

1. **Validazione campi** — se qualche campo obbligatorio è vuoto o il genere è null, si blocca e si mostra un errore.
2. **Parsing dell’anno** — il blocco try/catch intercetta la `NumberFormatException` se l’anno contiene caratteri non numerici. Se va bene, si carica il catalogo da disco.
3. **Inserimento o modifica?** — se `branoInModifica == null` si è in modalità inserimento: si controlla che il titolo non esista già (case-insensitive), si recupera l’email del proprietario dal `SessionManager` e si crea il nuovo brano. Se invece `branoInModifica != null` si è in modalità modifica: si salta il controllo del titolo e si itera il catalogo per trovare il brano da aggiornare, modificandone i campi con i setter.
4. **Persistenza** — in entrambi i casi si chiama `salvaCatalogo()` per scrivere su disco e si mostra il messaggio di conferma.

Questo approccio segue il principio DRY (Don’t Repeat Yourself): invece di due form separati per inserimento e modifica, un unico componente gestisce entrambi i casi con un semplice controllo sul riferimento.

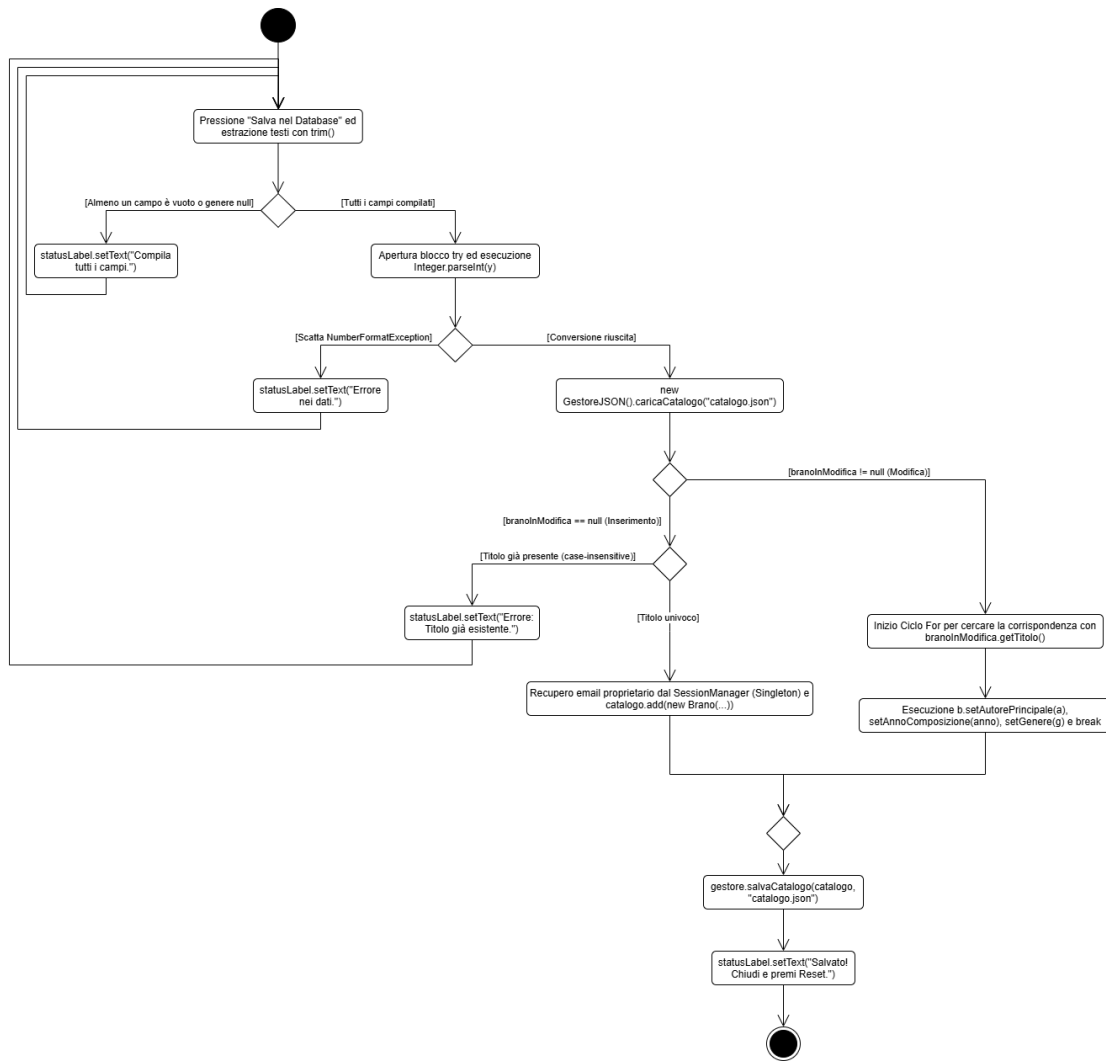


Figura 5.8: AD-03: Flusso di inserimento/modifica brano con gestione del riferimento `branoInModifica`.

5.2.5 AD-04: Ricerca con deep scanning ed eccezione custom

Il diagramma **AD-04** (Figura 5.9) è il corrispettivo procedurale di SD-03: mostra lo stesso algoritmo dal punto di vista del flusso di controllo. Corrisponde a `handleRicerca()` di `DashboardController`.

Il percorso

1. **Lettura dei criteri** — il Controller legge i quattro campi di filtro indipendenti; ogni criterio lasciato vuoto si comporta come un predicato sempre soddisfatto (`campo.isEmpty()`) e quindi non vincola la ricerca. Se tutti i campi sono vuoti, lo stream restituisce l'intero catalogo (non c'è un cortocircuito esplicito: i filtri vuoti lasciano semplicemente passare tutti i brani).
2. **Pipeline di filtraggio** — lo stream applica in AND i filtri valorizzati su titolo, autore e genere; per l'esecutore scende nei segmenti dei documenti associati (deep scanning tramite `flatMap`). I risultati vengono raccolti in `filtrati`.
3. **Risultati o eccezione** — se ci sono risultati, `tabellaBrani.setItems(filtrati)` aggiorna il viewport; se la lista è vuota, viene lanciata `BranoNonTrovatoException`,

il catch la intercetta, mostra il messaggio di errore in `errorSearchLabel` e svuota la tabella.

Entrambi i percorsi (risultati trovati oppure nessun risultato con relativa eccezione) convergono nello stesso Merge Node finale, garantendo che l'applicazione torni sempre in uno stato stabile.

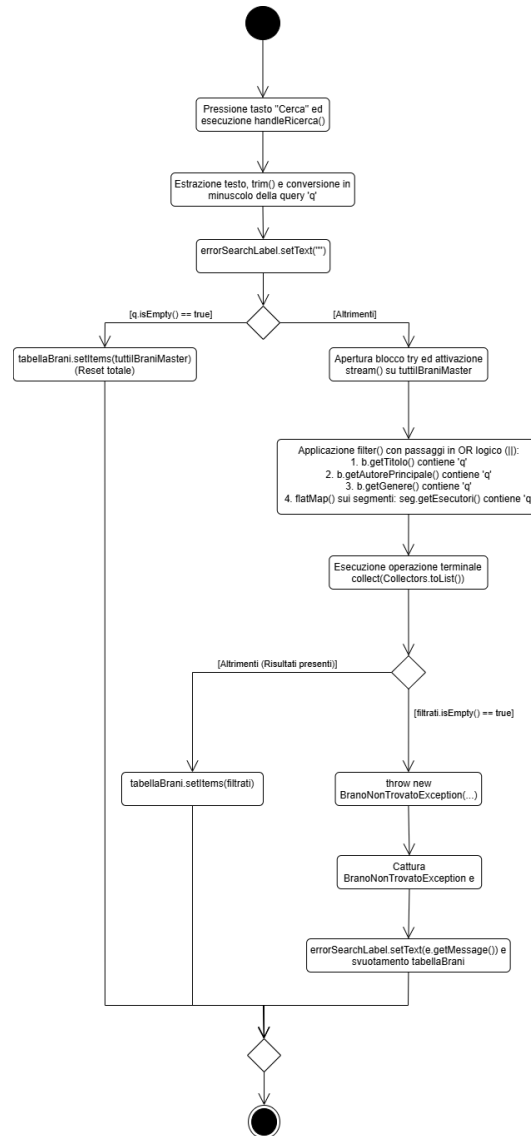


Figura 5.9: AD-04: Flusso dell'algoritmo di ricerca con gestione dell'eccezione custom.

Capitolo 6

Implementazione, Collaudo e Note Tecniche

6.1 Uso delle Java Stream API

In diversi punti del codice abbiamo preferito le Stream API ai classici cicli `for` annidati. Il motivo principale è la leggibilità: con uno stream si vede subito l'intenzione del codice (filtra, mappa, raccogli), mentre un ciclo imperativo richiede di leggere tutto il blocco per capire cosa fa.

Il caso più significativo è il motore di ricerca nella Dashboard. Lo stream non si limita a guardare i campi superficiali del **Branco** (titolo, autore, genere), ma con `flatMap()` entra anche nelle collezioni interne di documenti e nei loro segmenti di esecuzione, cercando il testo della query tra i musicisti che hanno partecipato a concerti o performance. Questo deep scanning in un ciclo imperativo annidato sarebbe molto più verboso e difficile da mantenere.

6.2 Test unitari con JUnit 5

Abbiamo scritto una suite di test automatici con **JUnit 5** articolata su cinque classi di test (circa undici metodi complessivi). L'esempio più significativo, riportato di seguito, è `NotaTemporaleTest`, dedicato al metodo `validaTimestamp(float durataMedia)`, che è il punto critico per la correttezza dei dati temporali. Si noti l'ordine dei parametri del costruttore di `NotaTemporale`, usato anche nei test: `testo`, `data`, `flag di ufficialità` (`boolean`), `email dell'autore` (`String`), `istante di inizio` e `istante di fine`.

```
package com.musicmanager.model.feedback;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

import java.util.Date;

public class NotaTemporaleTest {

    private NotaTemporale nota;
    private final float DURATA_MEDIA_TEST = 180.0f;

    @BeforeEach
```

```
public void setUp() {
    // Allocazione dell'oggetto prima di ogni test
    nota = new NotaTemporale(
        "Assolo di chitarra eccezionale",
        new Date(),
        false,
        "studente@univr.it",
        10.0f,
        60.0f
    );
}

@Test
public void testValidaTimestamp_Successo() {
    // Caso normale: intervallo dentro la durata del
    // media
    assertTrue(nota.validaTimestamp(DURATA_MEDIA_TEST),
        "Il timestamp dovrebbe essere valido per
        intervalli coerenti.");
}

@Test
public void testValidaTimestamp_ErroreInizioNegativo() {
    // Caso di errore: inizio negativo
    NotaTemporale notaInvalida = new NotaTemporale(
        "Inizio errato", new Date(), false, "
        studente@univr.it", -5.0f, 60.0f
    );
    assertFalse(notaInvalida.validaTimestamp(
        DURATA_MEDIA_TEST),
        "La validazione deve fallire se l'istante di
        inizio e negativo.");
}

@Test
public void testValidaTimestamp_ErroreInizioDopoFine() {
    // Caso di errore: inizio e fine invertiti
    NotaTemporale notaInvalida = new NotaTemporale(
        "Inversione temporale", new Date(), false, "
        studente@univr.it", 70.0f, 50.0f
    );
    assertFalse(notaInvalida.validaTimestamp(
        DURATA_MEDIA_TEST),
        "La validazione deve fallire se l'inizio supera l
        'istante di fine.");
}

@Test
public void testValidaTimestamp_ErroreFineSuperioreDurata
() {
    // Caso di errore: fine oltre la durata del file
}
```

```

        NotaTemporale notaInvalida = new NotaTemporale(
            "Oltre la durata", new Date(), false, "
            studente@univr.it", 10.0f, 200.0f
        );
        assertFalse(notaInvalida.validaTimestamp(
            DURATA_MEDIA_TEST),
            "La validazione deve fallire se l'istante di fine
            supera la durata del brano.");
    }
}

```

Listing 6.1: Test JUnit 5 per la validazione difensiva di NotaTemporale.

I quattro test coprono il caso felice e i tre principali casi di errore: timestamp di inizio negativo, inversione cronologica (inizio dopo fine), e fine oltre la durata effettiva del media. Il superamento di tutti i test garantisce che nessun dato temporale incoerente arrivi nel file JSON.

Oltre a `NotaTemporaleTest`, la suite comprende altre quattro classi: `CoreLogicTest` (validazione dell'intervallo temporale, mappatura dei metadati di un `SegmentoEsecuzione` e tracciamento dell'autore di un commento), `RicercaAlgoritmoTest` (la stessa logica di filtraggio della Dashboard verificata su un catalogo *mock*, sia per singolo criterio sia in combinazione AND), `GestoreJSONTest` (ciclo completo di salvataggio e ricaricamento del catalogo su file) e `IntegrazioneSystemTest` (test di integrazione sulla persistenza di un brano collegato all'email del suo proprietario). Tutti i test che accedono al disco operano su file temporanei dedicati, eliminati al termine di ogni esecuzione, così da non alterare i dati demo di consegna.

6.3 System Test funzionali

Oltre ai test unitari, abbiamo verificato i flussi principali in modo end-to-end, simulando il comportamento reale di un utente sull'interfaccia:

Tabella 6.1: System test per i flussi critici di MusicManager.

ID	Flusso	Cosa abbiamo verificato	Esito
ST-01	Ciclo registrazione e login	Registrazione → verifica attivo=false nel JSON → tentativo di login bloccato → approvazione admin → login riuscito.	Successo
ST-02	Inserimento con genere invalido	Apertura form → digitazione di un genere non presente nei dizionari → verifica che il salvataggio venga bloccato e il JSON non venga modificato.	Successo
ST-03	Anno non numerico	Campo Anno con caratteri alfa-numerici → intercettazione della <code>NumberFormatException</code> → alert grafico senza crash.	Successo

ID	Flusso	Cosa abbiamo verificato	Esito
ST-04	Commenti annidati	Selezione brano → apertura dettaglio → inserimento commenti e risposte → verifica della gerarchia nella TreeView.	Successo

6.4 Documentazione Javadoc

Tutto il codice sorgente è documentato con Javadoc. Per generare l'ipertesto HTML abbiamo usato il plugin Maven dedicato:

```
mvn javadoc:javadoc
```

Listing 6.2: Generazione Javadoc con Maven.

Il comando produce l'ipertesto HTML completo. Nel pacchetto di consegna la documentazione generata è inclusa nella cartella `src/javadoc/`, il cui punto di ingresso è `src/javadoc/index.html`. Rieseguendo il comando, la documentazione viene rigenerata dai sorgenti correnti nella cartella di build di Maven (`target/reports/apidocs/` oppure `target/site/apidocs/` a seconda della versione del plugin): è quindi opportuno rigenerarla dopo ogni modifica al codice, in modo che resti allineata ai sorgenti. I package documentati sono `com.musicmanager.model.users` (gerarchia degli utenti e meccanismi RBAC), `com.musicmanager.model.catalog` (classi del catalogo e system call per i player esterni) e `com.musicmanager.model.feedback` (validazione temporale e struttura Composite dei commenti). Abbiamo usato i tag standard (`@param`, `@return`, `@throws`) per documentare in modo preciso gli argomenti, i valori di ritorno e le eccezioni di ogni metodo pubblico.

Appendice A

Manuale d'Installazione e Note di Esecuzione

Questa appendice descrive la struttura del pacchetto di consegna e come avviare l'applicazione.

A.1 Struttura dell'archivio

```
MusicManager_Consegna/  
|-- src/                                # Codice sorgente  
|   |-- main/                          # applicazione (Main) e  
|       risorse (FXML, media)  
|   |-- test/                          # test JUnit 5  
|   '-- javadoc/                       # Documentazione HTML (  
|       aprire index.html)  
|-- media/                             # File multimediali demo (mp3  
|   , mp4, pdf)  
|-- pom.xml                           # Dipendenze e configurazione  
|   di build Maven  
|-- catalogo.json                     # Database dei brani (gia  
|   popolato)  
|-- utenti.json                       # Registro degli utenti (gia  
|   popolato)  
|-- dizionari.json                   # Vocabolari controllati (gia  
|   popolato)  
|-- Documentazione.pdf               # Questa relazione in PDF  
'-- README.md                        # Istruzioni rapide
```

Listing A.1: Struttura del pacchetto di distribuzione.

I file JSON stanno nella root del progetto (non dentro **resources**) perché, se finissero dentro il JAR compilato da Maven, diventerebbero read-only e l'applicazione andrebbe in crash alla prima scrittura. Per lo stesso motivo i file JSON e la cartella **media/** vengono referenziati tramite percorsi relativi: i comandi e l'esecuzione vanno quindi lanciati dalla cartella radice del progetto (vedi la sezione successiva), altrimenti i dati e i media non vengono trovati.

A.2 Prerequisiti

- **JDK 17 o superiore** (testato anche con JDK 21);
- **Apache Maven 3.8 o superiore**, necessario per scaricare le dipendenze ed eseguire i goal `javafx:run`, `test` e `javadoc:javadoc`;
- **IntelliJ IDEA** (consigliato) o Eclipse con supporto Maven, in alternativa alla riga di comando;
- **Connessione a Internet** al primo avvio, per il download da Maven Central delle dipendenze dichiarate nel `pom.xml` (Google Gson 2.10.1, JavaFX 19.0.2 `controls+fxml`, JUnit 5.9.2);
- **Ambiente desktop con interfaccia grafica**, con supporto alle system call di `java.awt.Desktop` per l'apertura dei player esterni: l'applicazione non funziona in modalità *headless* o solo terminale.

A.3 Come avviare il progetto

Nota importante sulla directory di lavoro. Tutti i comandi vanno eseguiti dalla cartella radice `MusicManager_Consegna/` (quella che contiene `pom.xml`, i tre file `.json` e la cartella `media/`), perché i percorsi ai dati e ai media sono relativi. Da IDE, verificare che la *working directory* della configurazione di esecuzione sia la radice del progetto (è il default in IntelliJ IDEA).

A.3.1 Avvio da riga di comando (CLI)

Posizionarsi nella cartella radice del progetto ed eseguire:

```
mvn clean compile      # Compilazione
mvn javafx:run         # Avvio dell'app (entry point com.
                        musicmanager.main.Main)
mvn test               # Esecuzione dei test JUnit 5
mvn javadoc:javadoc    # (Opzionale) Rigenerazione del Javadoc
```

Il goal `javafx:run` gestisce automaticamente il ciclo di vita dei moduli JavaFX e le dipendenze dichiarate nel `pom.xml`. Il goal `test` esegue i test JUnit 5 (l'engine `junit-jupiter-engine` è già incluso tra le dipendenze).

A.3.2 Avvio da IDE

1. **Aprire il progetto:** in IntelliJ scegliere *Open* e puntare alla cartella `MusicManager_Consegna/`. L'IDE riconosce automaticamente il `pom.xml` e scarica JavaFX, Google Gson e JUnit.
2. **Eseguire i test:** tasto destro su `src/test/java` e selezionare *Run All Tests* per verificare gli scudi difensivi con la suite JUnit 5.
3. **Avviare l'app:** aprire `Main.java` nel package `com.musicmanager.main` ed eseguire il metodo `main()` con il tasto Run dell'IDE. La classe `Main` funge da launcher statico e delega l'avvio a `MusicManagerGUI`, consentendo l'inizializzazione corretta dei moduli JavaFX in un progetto non modulare.

A.4 Credenziali per il collaudo

I database JSON vengono consegnati già popolati con dati demo. Gli account disponibili sono:

Ruolo	Email	Password
Amministratore	admin@musicmanager.it	admin123
Autore/Interprete	angelo@musica.it	angelo

Tabella A.1: Credenziali di test.

L'account Amministratore è un account di sistema predefinito, gestito direttamente dal `LoginController` (non è presente in `utenti.json`). L'account di Angelo è invece registrato in `utenti.json` con `attivo = true` ed è di tipo Autore/Interprete. Con l'Amministratore si possono approvare utenti, gestire i dizionari e moderare i commenti. Con l'account di Angelo si accede alle funzioni core: inserimento brani, riproduzione file, interazione social con i commenti e, in quanto autore, inserimento di commenti contrassegnati come ufficiali.